

OOP Concepts in C++

Learning Material

November 18, 2024

Introduction to OOP Concepts

Object-Oriented Programming (OOP) in C++ revolves around four core concepts: **Abstraction**, **Encapsulation**, **Inheritance**, and **Polymorphism**. This document covers these concepts with examples and implementations.

Abstraction

Abstraction is the concept of hiding the complex implementation details and showing only the essential features of the object. It can be achieved using abstract classes and interfaces.

Example

```
1 // Abstract Class Example
2 #include <iostream>
3 using namespace std;
4
5 class AbstractBase {
6 public:
7     virtual void display() = 0;    // Pure virtual function
8 };
9
10 class Derived : public AbstractBase {
11 public:
12     void display() override {
13         cout << "Derived class implementation of display" << endl
14         ;
15     }
16 };
17
18 int main() {
19     Derived d;
20     d.display();
21     return 0;
22 }
```

Encapsulation

Encapsulation is the concept of wrapping data and methods that operate on the data within a single unit (a class). It restricts direct access to some of an object's components, preventing accidental interference and misuse of the data.

Example

```
1 // Encapsulation Example
2 #include <iostream>
3 using namespace std;
4
5 class Person {
6 private:
7     int age;
8     string name;
9
10 public:
11     void setName(string n) {
12         name = n;
13     }
14
15     string getName() {
16         return name;
17     }
18
19     void setAge(int a) {
20         if (a > 0) {
21             age = a;
22         }
23     }
24
25     int getAge() {
26         return age;
27     }
28 };
29
30 int main() {
31     Person person;
32     person.setName("John");
33     person.setAge(30);
34     cout << "Name: " << person.getName() << "\\nAge: " << person.
35         getAge() << endl;
36
37     return 0;
38 }
```

Polymorphism

Polymorphism allows methods to do different things based on the object it is acting upon, even though they share the same name. It comes in two forms: **Compile-Time** and **Runtime Polymorphism**.

Compile-Time Polymorphism (Function Overloading)

```
1 // Function Overloading Example
2 #include <iostream>
3 using namespace std;
4
5 class Print {
6 public:
7     void display(int i) {
8         cout << "Printing int: " << i << endl;
9     }
10
11     void display(double f) {
12         cout << "Printing float: " << f << endl;
13     }
14
15     void display(string s) {
16         cout << "Printing string: " << s << endl;
17     }
18 };
19
20 int main() {
21     Print print;
22     print.display(5);
23     print.display(3.14);
24     print.display("Hello");
25
26     return 0;
27 }
```

Operator Overloading

```
1 // Operator Overloading Example
2 #include <iostream>
3 using namespace std;
4
5 class Complex {
6 public:
7     float real;
8     float imag;
9     Complex operator + (const Complex& c) {
10         Complex temp;
11         temp.real = real + c.real;
```

```

12         temp.imag = imag + c.imag;
13         return temp;
14     }
15 };
16
17 int main() {
18     Complex c1, c2, c3;
19     c1.real = 1.2; c1.imag = 3.4;
20     c2.real = 5.6; c2.imag = 7.8;
21
22     c3 = c1 + c2; // Using the overloaded '+' operator
23     cout << "Result: " << c3.real << " + " << c3.imag << "i" <<
        endl;
24
25     return 0;
26 }

```

Runtime Polymorphism (Method Overriding)

```

1  // Runtime Polymorphism Example
2  #include <iostream>
3  using namespace std;
4
5  // Base class
6  class Base {
7  public:
8      virtual void show() {
9          cout << "Base class show() method" << endl;
10     }
11
12     void display() {
13         cout << "Base class display() method" << endl;
14     }
15 };
16
17 // Derived class
18 class Derived : public Base {
19 public:
20     void show() override {
21         cout << "Derived class show() method" << endl;
22     }
23
24     void display() {
25         cout << "Derived class display() method" << endl;
26     }
27 };
28
29 int main() {
30     Base* basePointer;
31     Derived derivedObject;

```

```

32
33     basePointer = &derivedObject;
34
35     basePointer->show();    // Calls Derived class show()
36     basePointer->display(); // Calls Base class display()
37
38     return 0;
39 }

```

Inheritance

Inheritance allows a class (**derived class**) to inherit properties and behavior (methods) from another class (**base class**). This promotes code reuse.

Example

```

1 // Inheritance Example
2 #include <iostream>
3 using namespace std;
4
5 class Animal {
6 public:
7     void eat() {
8         cout << "This animal eats food." << endl;
9     }
10 };
11
12 class Dog : public Animal {
13 public:
14     void bark() {
15         cout << "The dog barks." << endl;
16     }
17 };
18
19 int main() {
20     Dog dog;
21     dog.eat(); // Inherited method
22     dog.bark();
23
24     return 0;
25 }

```

Constructors and Destructors

Constructors are special class functions that perform initialization of every object. They are invoked when an object is created. Destructors are used to clean up the resources allocated to the object when it is destroyed.

Example

```
1 // Constructors and Destructors Example
2 #include <iostream>
3 using namespace std;
4
5 class Person {
6 public:
7     string name;
8
9     // Constructor
10    Person(string n) {
11        name = n;
12        cout << name << " is created" << endl;
13    }
14
15    // Destructor
16    ~Person() {
17        cout << name << " is destroyed" << endl;
18    }
19 };
20
21 int main() {
22     Person person1("John");
23     Person person2("Doe");
24
25     return 0;
26 }
```

Templates

Templates are a powerful feature in C++ that allow you to write generic and reusable code. Template functions operate with any data type. A template is defined using the `template` keyword followed by a template parameter list inside angle brackets `<>`.

Template Functions

```
1 // Template Functions Example
2 #include <iostream>
3 using namespace std;
4
5 template <typename T>
6 T getMax(T a, T b) {
7     return (a > b) ? a : b;
8 }
9
10 template <typename T>
11 T getMin(T a, T b) {
12     return (a < b) ? a : b;
```

```

13 }
14
15 int main() {
16     cout << "Max of 3 and 7: " << getMax(3, 7) << endl;
17     cout << "Min of 3 and 7: " << getMin(3, 7) << endl;
18
19     return 0;
20 }

```

Generic Class (Stack Example)

```

1 // Generic Stack Example
2 #include <iostream>
3 using namespace std;
4
5 template <typename T>
6 class Stack {
7     T* arr;
8     int top;
9     int capacity;
10 public:
11     Stack(int size = 10);
12     ~Stack();
13     void push(T element);
14     T pop();
15     T peek();
16     bool isEmpty();
17     bool isFull();
18 };
19
20 template <typename T>
21 Stack<T>::Stack(int size) {
22     arr = new T[size];
23     capacity = size;
24     top = -1;
25 }
26
27 template <typename T>
28 Stack<T>::~~Stack() {
29     delete[] arr;
30 }
31
32 template <typename T>
33 void Stack<T>::push(T element) {
34     if (isFull()) {
35         cout << "Stack Overflow" << endl;
36         return;
37     }
38     arr[++top] = element;
39 }

```

```

40
41 template <typename T>
42 T Stack<T>::pop() {
43     if (isEmpty()) {
44         cout << "Stack Underflow" << endl;
45         return T(); // Return default value
46     }
47     return arr[top--];
48 }
49
50 template <typename T>
51 T Stack<T>::peek() {
52     if (isEmpty()) {
53         cout << "Stack is Empty" << endl;
54         return T(); // Return default value
55     }
56     return arr[top];
57 }
58
59 template <typename T>
60 bool Stack<T>::isEmpty() {
61     return top == -1;
62 }
63
64 template <typename T>
65 bool Stack<T>::isFull() {
66     return top == capacity - 1;
67 }
68
69 int main() {
70     Stack<int> intStack;
71     intStack.push(1);
72     intStack.push(2);
73     intStack.push(3);
74     cout << "Top of int stack: " << intStack.peek() << endl;
75     cout << "Popped from int stack: " << intStack.pop() << endl;
76
77     Stack<string> stringStack;
78     stringStack.push("Hello");
79     stringStack.push("World");
80     cout << "Top of string stack: " << stringStack.peek() << endl
81         ;
82     cout << "Popped from string stack: " << stringStack.pop() <<
83         endl;
84     return 0;
85 }

```

Generic Class (Matrix Example)


```

1 // Generic Matrix Example
2 #include <iostream>
3 using namespace std;
4
5 template<class T>
6 class Matrix {
7     T** data;
8     int rows, cols;
9 public:
10    Matrix(int r, int c);
11    ~Matrix();
12    void setElement(int r, int c, T value);
13    T getElement(int r, int c);
14    void display();
15 };
16
17 template<class T>
18 Matrix<T>::Matrix(int r, int c) {
19     rows = r;
20     cols = c;
21     data = new T*[rows];
22     for (int i = 0; i < rows; ++i) {
23         data[i] = new T[cols];
24     }
25 }
26
27 template<class T>
28 Matrix<T>::~~Matrix() {
29     for (int i = 0; i < rows; ++i) {
30         delete[] data[i];
31     }
32     delete[] data;
33 }
34
35 template<class T>
36 void Matrix<T>::setElement(int r, int c, T value) {
37     if (r >= 0 && r < rows && c >= 0 && c < cols) {
38         data[r][c] = value;
39     }
40 }
41
42 template<class T>
43 T Matrix<T>::getElement(int r, int c) {
44     if (r >= 0 && r < rows && c >= 0 && c < cols) {
45         return data[r][c];
46     }
47 }
48
49 template<class T>
50 void Matrix<T>::display() {

```

```

51     for (int i = 0; i < rows; ++i) {
52         for (int j = 0; j < cols; ++j) {
53             cout << data[i][j] << " ";
54         }
55         cout << endl;
56     }
57 }
58
59 int main() {
60     Matrix<int> intMatrix(2, 2);
61     intMatrix.setElement(0, 0, 1);
62     intMatrix.setElement(0, 1, 2);
63     intMatrix.setElement(1, 0, 3);
64     intMatrix.setElement(1, 1, 4);
65     intMatrix.display();
66
67     Matrix<double> doubleMatrix(2, 2);
68     doubleMatrix.setElement(0, 0, 1.1);
69     doubleMatrix.setElement(0, 1, 2.2);
70     doubleMatrix.setElement(1, 0, 3.3);
71     doubleMatrix.setElement(1, 1, 4.4);
72     doubleMatrix.display();
73
74     return 0;
75 }

```

Structures and Functions

Structures in C++ are user-defined data types that allow grouping of variables of different types. They can be passed to functions by value or by reference.

Passing Structure by Reference

```

1  // Structure as Parameter (Pass by Reference)
2  #include<iostream>
3  using namespace std;
4
5  struct person {
6      string name;
7      int age;
8  };
9
10 int updateAge(person *p) {
11     p->age += 1;
12 }
13
14 int main() {
15     person p1 = {"Shahid", 13};
16     updateAge(&p1);

```

```

17     cout << "Updated Age: " << p1.age << endl;
18     return 0;
19 }

```

Passing Structure by Value

```

1 // Structure as Parameter (Pass by Value)
2 #include <iostream>
3 using namespace std;
4
5 struct Person {
6     string name;
7     int age;
8 };
9
10 void displayPerson(Person p) {
11     cout << "Name: " << p.name << ", Age: " << p.age << endl;
12 }
13
14 int main() {
15     Person person1 = {"Alice", 25};
16     displayPerson(person1); // Pass by value
17     return 0;
18 }

```

Returning Arrays from Functions

C++ allows functions to return pointers to dynamically created arrays.

Example

```

1 // Returning Arrays
2 #include<iostream>
3 using namespace std;
4
5 int* CreateArray(int size) {
6     int* arr = new int[size];
7     for (int i = 0; i < size; i++) {
8         arr[i] = i + 1;
9     }
10    return arr;
11 }
12
13 int main() {
14     int n = 5;
15     // Get the array from the function
16     int* MyArray = CreateArray(n);
17

```

```

18 // Print Array
19 cout << "Array elements are :" << endl;
20 for (int i = 0; i < n; i++) {
21     cout << MyArray[i] << " ";
22 }
23 cout << endl;
24
25 delete[] MyArray; // Deallocate memory
26 return 0;
27 }

```

Dynamic Memory Allocation in C++

Dynamic memory allocation allows creating objects and arrays at runtime, providing more flexibility.

Example

```

1 // Dynamic Memory Allocation
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // Dynamic allocation of a single integer
7     int* ptr = new int;
8     *ptr = 42;
9     cout << "Value pointed to by ptr: " << *ptr << endl;
10    delete ptr; // Deallocates memory
11
12    // Dynamic allocation of an array of integers
13    int size;
14    cout << "Enter the size of the array: ";
15    cin >> size;
16
17    int* arr = new int[size];
18    for (int i = 0; i < size; ++i) {
19        arr[i] = i * 2;
20    }
21
22    cout << "Array elements: ";
23    for (int i = 0; i < size; ++i) {
24        cout << arr[i] << " ";
25    }
26    cout << endl;
27
28    delete[] arr; // Deallocates memory for the array
29
30    return 0;
31 }

```

Memory Leaks in Data Structures and Algorithms (DSA)

A **memory leak** occurs when a program dynamically allocates memory (e.g., using `new` in C++ or `malloc` in C) but fails to properly release that memory after it is no longer needed. This can lead to excessive memory usage, potentially causing the program or system to crash due to insufficient memory.

Common Causes of Memory Leaks in DSA

1. **Forgetting to Deallocate Memory:** Dynamically allocated memory is not explicitly released using `delete` (C++) or `free` (C).
2. **Dangling Pointers:** Memory is deallocated, but a pointer to the memory location still exists, leading to undefined behavior.
3. **Circular References:** In data structures like graphs, reference cycles prevent memory from being freed by garbage collection.
4. **Overwriting Pointers:** Assigning a new address to a pointer without freeing its previous memory block.

Example of a Memory Leak

```
1 // Memory Leak Example
2 #include <iostream>
3 using namespace std;
4
5 void memoryLeakExample() {
6     int* ptr = new int(5); // Memory allocated dynamically
7     cout << "Value: " << *ptr << endl;
8     // Forget to deallocate memory
9 }
10
11 int main() {
12     memoryLeakExample();
13     return 0;
14 }
```

Explanation

In the above example, the memory allocated to `ptr` is never deallocated, causing a memory leak.

Impact of Memory Leaks

1. **Increased Memory Usage:** The program consumes more memory over time, reducing available memory for other processes.

2. **Performance Degradation:** Continuous memory leaks can lead to slower execution or program crashes.
3. **Out of Memory (OOM):** In extreme cases, the system may run out of memory, causing the program or system to crash.

How to Overcome Memory Leaks

1. **Use Smart Pointers (C++):** Replace raw pointers with smart pointers like `std::unique_ptr` or `std::shared_ptr`, which automatically manage memory.

```
1 // Smart Pointer Example
2 #include <iostream>
3 #include <memory>
4 using namespace std;
5
6 void smartPointerExample() {
7     auto ptr = make_unique<int>(5); // Memory managed
8     cout << "Value: " << *ptr << endl;
9 }
10
11 int main() {
12     smartPointerExample();
13     return 0;
14 }
```

2. **Manually Free Allocated Memory:** Always pair `new` with `delete` in C++ or `malloc` with `free` in C.

```
1 // Manual Deallocation Example
2 void manualDeallocationExample() {
3     int* ptr = new int(10);
4     cout << "Value: " << *ptr << endl;
5     delete ptr; // Free the allocated memory
6 }
```

3. **Use Destructors in Classes:** For custom data structures, define destructors to release dynamically allocated memory.

```
1 // Destructor Example
2 class Node {
3 public:
4     int data;
5     Node* next;
6
7     Node(int val) : data(val), next(nullptr) {}
8     ~Node() {
9         delete next; // Recursively delete nodes
10    }
11 };
```

4. **Track Allocated Memory:** Maintain a record of all dynamic memory allocations and deallocations. Use tools like Valgrind (Linux) or AddressSanitizer to detect memory leaks.
5. **Avoid Circular References:** For data structures like graphs or doubly linked lists, use weak references (`std::weak_ptr`) where appropriate to break cycles.

Example: Properly Freeing Memory in a Linked List

```
1 // Properly Freeing Memory in a Linked List
2 #include <iostream>
3 using namespace std;
4
5 struct Node {
6     int data;
7     Node* next;
8
9     Node(int val) : data(val), next(nullptr) {}
10 };
11
12 void deleteList(Node* head) {
13     while (head != nullptr) {
14         Node* temp = head;
15         head = head->next;
16         delete temp; // Free memory
17     }
18 }
19
20 int main() {
21     Node* head = new Node(1);
22     head->next = new Node(2);
23     head->next->next = new Node(3);
24
25     cout << "Linked list created." << endl;
26
27     // Free the allocated memory
28     deleteList(head);
29
30     cout << "Memory freed." << endl;
31     return 0;
32 }
```

Output

Linked list created.
Memory freed.

Explanation

The `deleteList` function ensures that all nodes in the linked list are properly deallocated.

Best Practices to Avoid Memory Leaks

1. Always match `new` with `delete` and `malloc` with `free`.
2. Use smart pointers or garbage collectors where applicable.
3. Regularly check for memory leaks using tools like:
 - **Valgrind (Linux):** Detects memory leaks and invalid memory usage.
 - **AddressSanitizer:** A fast memory error detector.
4. Define destructors in custom classes to release resources.
5. Avoid circular references in data structures.
6. Test and review dynamic memory management code thoroughly.

Conclusion

Memory leaks can severely impact the performance of applications in DSA, especially when dealing with dynamic data structures like trees, graphs, and linked lists. By following proper memory management practices, using tools for leak detection, and leveraging smart pointers, memory leaks can be effectively avoided.